

PulsePoint

A symptom triage and care-finding app for Android

COMP826 Mobile Systems Development

Milestone 2 - Implementation and Evaluation

Jaydev Patel

Student ID 26324224

Auckland University of Technology

1 October 2026

github.com/ijaydevpatel/PulsePointApp

Table of Contents

1 System Overview	4
1.1 Introduction and existing work	4
1.2 Compiling and running the project	4
1.3 Role and key features.....	4
2 System Artefacts: features, evidence and demo	5
2.1 Symptom check and triage.....	5
2.2 Finding care nearby.....	5
2.3 Records, profile and settings.....	5
2.4 The feature that was cut	6
3 Design, Implementation and Process	6
3.1 Technology stack and development tools	6
3.2 Logical decomposition	6
3.3 The class model.....	9
3.4 Runtime view	14
3.5 Physical view	18
3.6 Variations from the Milestone 1 design.....	21
3.7 Implementation reflection	21
3.8 Development process	21
3.9 Process reflection.....	26
4 System Evaluation.....	26
4.1 Evaluation framework and benchmark devices.....	26
4.2 Evaluation against functional criteria	26
4.3 Evaluation against non-functional criteria.....	27
4.4 Industry evaluation methods	28
4.5 Reflection on the evaluation results	28
5 Conclusion and Recommendations	28
5.1 Strengths	28
5.2 Limitations.....	28
5.3 Future work.....	29
5.4 Conclusion.....	29
References	29

List of Figures

- Figure 1: Home tab and symptom entry
- Figure 2: Triage result with reasoning and red-flag notice
- Figure 3: Care tab: full-screen map with labelled pins
- Figure 4: Facility callout and the Directions handoff
- Figure 5: Records overlay and an expanded record
- Figure 6: Profile and Settings in dark mode
- Figure 7: Layered architecture
- Figure 8: Component and interface view
- Figure 8a: Screen state and event handling

Figure 8b: View layer imports

Figure 9: Class diagram: storage package

Figure 9a: All storage interface realizations

Figure 9b: History entry value relationships

Figure 10: Class diagram: domain package

Figure 10a: Assessment use case relationships

Figure 11: Class diagram: care package

Figure 12: Symptom check sequence

Figure 12a: Triage band and result status logic

Figure 12b: Optional remote analysis sequence

Figure 13: Screen navigation map

Figure 13a: Facility API query flow

Figure 13b: Facility query geographic scope

Figure 14: Deployment view

Figure 14a: Earlier deployment from verified Git history

Figure 14b: Remote API adapter environment

Figure 14c: Local SQLite schema

Figure 15: How Mobile-D and Scrum fit together

Figure 15a: Development approach comparison

Figure 15b: Verified Git change history

Figure 16: The build loop inside one sprint

Figure 17: Milestone 2 schedule

Figure 17a: Expected outcomes and evidence

1 System Overview

1.1 Introduction and existing work

PulsePoint is an Android prototype for checking symptom urgency and finding nearby care. It combines local rules with a facility map; clinical accuracy is not established.

Scope includes triage, medicines/interactions, Care, records, profiles and AI chat. Clinician consultation, history sync, clinical validation and iOS are excluded.

Clerk authenticates users. ApiClient sends selected symptoms and free text to remote analysis. Episode history and activity logs stay local; no working history-sync service exists. Backend hosting is unverified.

1.2 Compiling and running the project

Repository: <https://github.com/ijaydevpatel/PulsePointApp>

1. Install Node, npm, JDK 17 and Android Studio with the SDK required by Expo SDK 54. The checked native build configures minimum API 29.
2. Clone the repository and run npm install. Copy .env.example to .env; supply the Clerk publishable key and API origin. Keep server credentials out of the app.
3. Run npm run prebuild, then npm run android with a device or emulator connected. android/ is generated and git-ignored.
4. Run npm run verify for types, boundaries, contrast and tests. For a release APK, run ./gradlew assembleRelease inside android/. The checked release configuration uses the debug signing key; production distribution needs separate signing.

Only the API origin and Clerk publishable key belong in client configuration. Credential-pattern tests cannot establish that all Git history is secret-free.

1.3 Role and key features

I handled development and related roles. Revision 5a8fa2b contains 65 application TypeScript files, 21 classes and 215 static module links, plus Android entry classes.

ID	Requirement	Actual source	Status
FR1	Clerk sign-in	AuthScreen; App.tsx	Present
FR2	Symptom entry	TriageScreen; symptomCatalogue	Present
FR3	Local urgency band / red flags	assessSymptoms; RuleClassifier; ResultScreen	Present; unvalidated clinically
FR4	Medicine lookup	MedicinesScreen; remoteServices	Present
FR5	Nearby-care map	CareScreen; OverpassFacilities	Present; network dependent
FR6	External directions	CareScreen.openDirections	Present

ID	Requirement	Actual source	Status
FR7	Local history	DurableEpisodeStore; RecordsScreen	Present; encryption unverified
FR8	Health profile	ProfileScreen; RemoteProfile	Present; remote
FR9	Clinician consultation	No clinician service	Out of scope
FR10	Medicine interactions	checkInteractions; CollisionScreen	Reachable
FR11	AI chat	ChatScreen; RemoteChat	Reachable; not a clinician

2 System Artefacts: features, evidence and demo

The demonstration should cover sign-in, assessment, Care, Directions, Records, interactions and AI Doctor. Screenshots and recording remain to be inserted.

[ADD DEMONSTRATION LINK AND DURATION BEFORE SUBMISSION]

2.1 Symptom check and triage

TriageScreen collects symptoms, severity, duration and free text. ResultScreen gives band advice, confidence information and red flags. Non-escalated display waits for remote completion or 12 seconds; escalation bypasses that wait. There is no rating control.

[INSERT SCREENSHOT - Figure 1: Home tab and the symptom questionnaire]

[INSERT SCREENSHOT - Figure 2: Triage result with reasoning and red-flag notice]

2.2 Finding care nearby

Care queries three expanding Overpass boxes, merges facilities and sorts distance progressively. Warm cached results are refreshed. Selecting a facility opens its sheet; Directions opens an external maps application.

Four mirrors are raced and unused requests aborted. MapLibre retries label placement. Neither a category fallback name nor a map entry proves a facility is open or suitable.

[INSERT SCREENSHOT - Figure 3: Care tab: full-screen map with labelled pins]

[INSERT SCREENSHOT - Figure 4: Facility callout and the Directions handoff]

2.3 Records, profile and settings

Records opens from Profile/Settings. DurableEpisodeStore uses SQLite or substitutes memory after initialization failure; fallback data vanishes on restart. Encryption support is unverified. Profile storage is remote.

[INSERT SCREENSHOT - Figure 5: Records tab and an expanded record]

[INSERT SCREENSHOT - Figure 6: Profile and Settings in dark mode]

2.4 The feature that was cut

AI Doctor uses ChatScreen; it is not clinician consultation. Medicines supports interaction checks. News/check-in route cases lack entries from rendered tabs and are not counted as demonstrated features.

3 Design, Implementation and Process

3.1 Technology stack and development tools

The stack choice followed four constraints.

- Project factors: Expo, React Native and TypeScript suited one Android developer. Prebuild exposes Gradle. Flutter would add Dart; native Kotlin would require separate application/UI work.
- Historical factors: existing PulsePoint work used TypeScript. Keeping the language reduced duplicated conventions. This app uses Clerk, not verified reuse of website auth routes.
- Environmental factors: Overpass avoids an APK-bundled facility list but depends on network and coverage. Mirror racing and caching do not provide full offline service.
- Business factors: MapLibre avoids a proprietary rendering SDK. External tile/facility services still have operating constraints; commercial deployment needs attribution, service terms and cost estimates.

VS Code provides editing; Android Studio provides Gradle, device tools and Logcat. Git/GitHub record changes. The table lists directly used Expo packages.

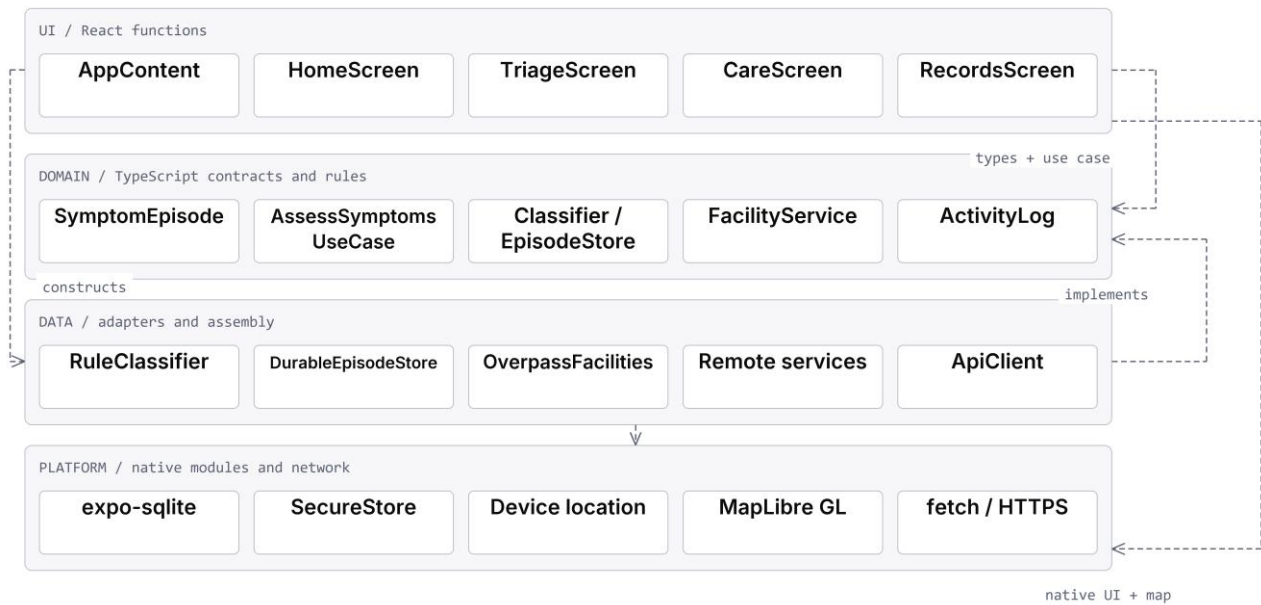
Imported directly by the app	What it does here
expo-sqlite 16.0.10	SQLite episode/activity store; encryption unverified
expo-secure-store 15.0.8	Holds the database key in the Android keystore
expo-location 19.0.8	Two-stage position fix with mocked-reading rejection
expo-crypto 15.0.9	Identifiers and key material
expo-splash-screen 31.0.13	Holds the splash until the first screen is ready
expo-font 14.0.12	Loads the type ramp before first paint
expo-haptics 15.0.8	Confirmation feedback on primary actions
expo-web-browser 15.0.11	Opens facility websites outside the app

Installed packages do not prove active features. Assess unused dependencies before removal, then rebuild and test. No removal result is claimed.

3.2 Logical decomposition

Clean Architecture governs inward dependencies (Martin, 2017). React functions, hooks and callbacks organise this UI; no ViewModel class exists. Figures 7/8 show layers/interfaces; 8a/8b show state/view imports.

Layered architecture and import direction



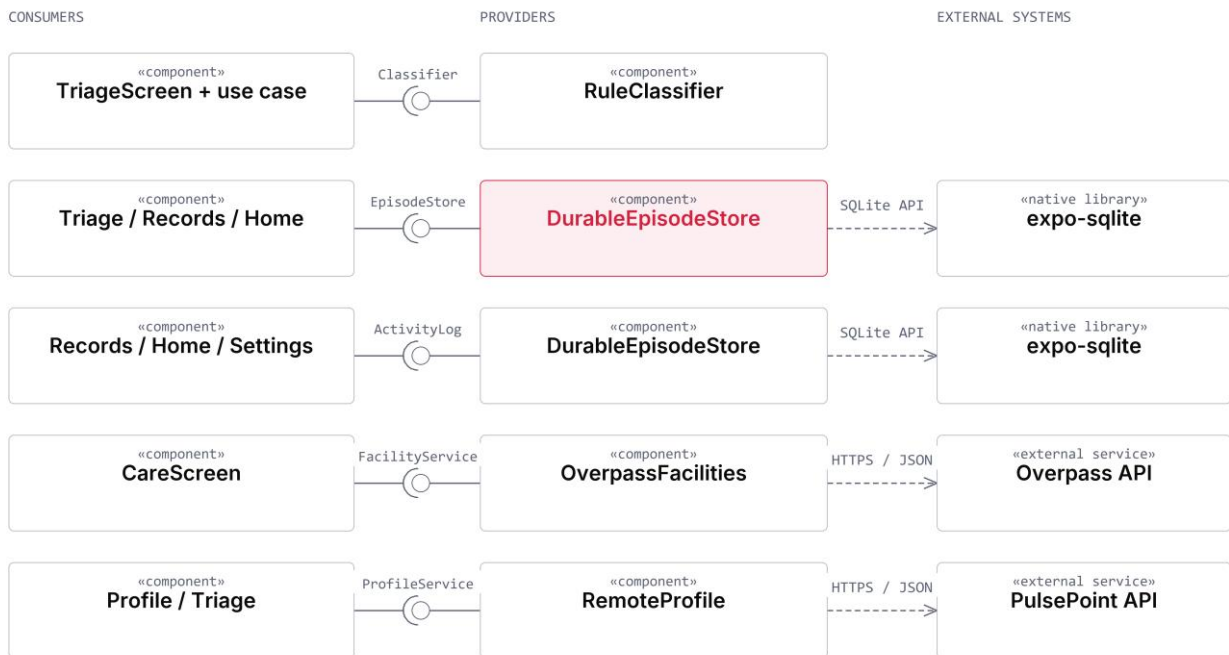
Arrows point from importer to dependency. **AppContent** assembles concrete adapters; screens also use catalogue, cache and location helpers. Domain files import only other domain files. The platform row shows adapter implementation dependencies, not an extra application package.

Figure 7: Layered architecture

The boundary check passes. It scans direct domain-file patterns, not transitive imports. The separate static audit records wider module relationships.

Figure 8 places provided interfaces at providers and required sockets at consumers. **AppContent** assembles dependencies; assessment mediates the classifier call.

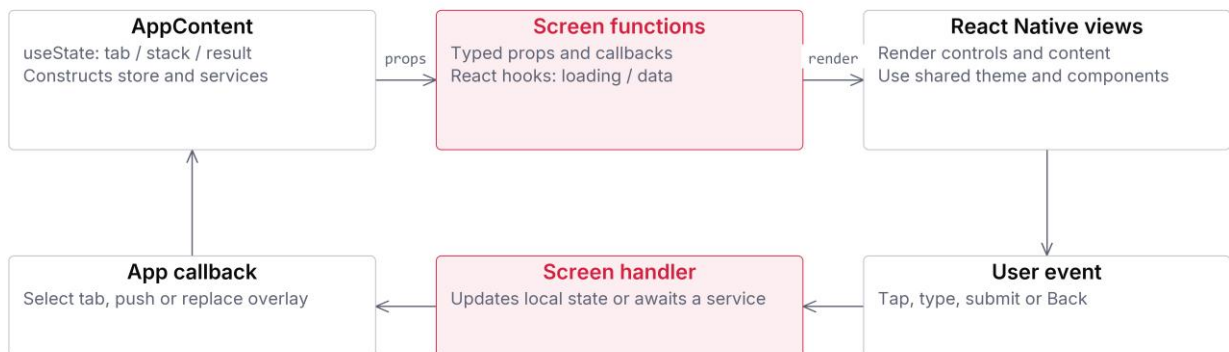
Component and interface view



Socket = required interface; ball = provided interface. Triage calls Classifier and EpisodeStore through AssessSymptomsUseCase. Services are injected by AppContent / createServices. SQLite access is delegated; DurableEpisodeStore does not hold a database connection.

Figure 8: Component and interface view

Screen state and event handling

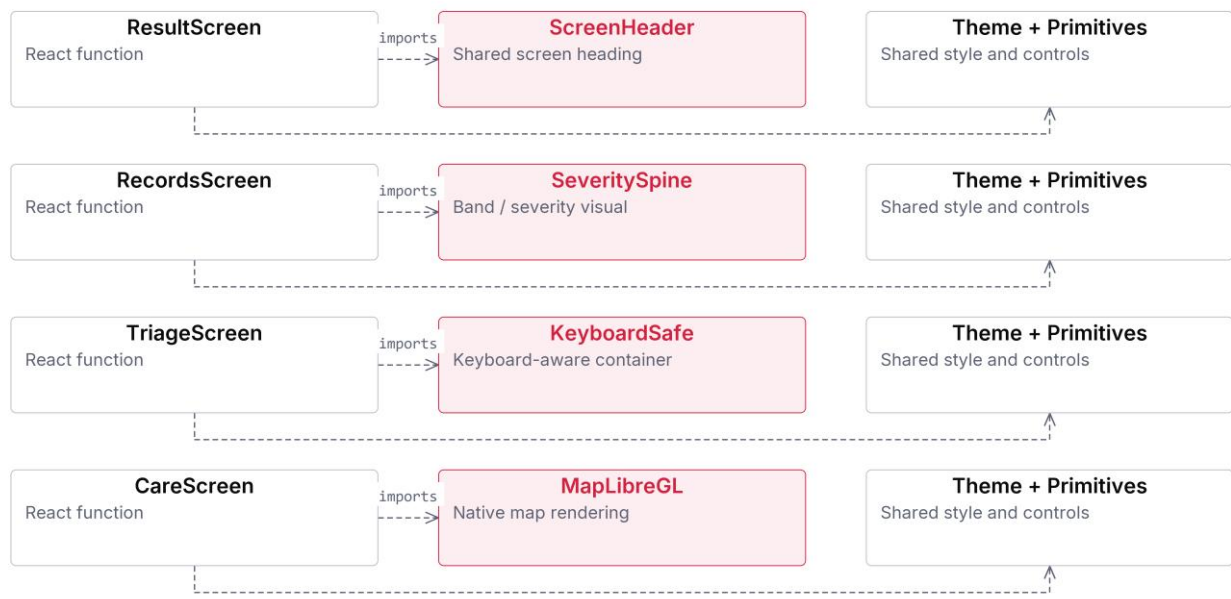


This project defines screen functions and React hooks, not a Presenter or ViewModel class. The event loop returns changed state to the view. Screens can also update their own hook state after a service reply. Domain use cases and adapters are separate from visual components.

Figure 8a: Screen state and event handling

State and callbacks return user events to the parent; this is the actual React arrangement.

View layer imports



Dashed arrows mean imports. Screens remain React functions rather than UML classes. MapLibreGL is an external module; other shown dependencies are local UI modules.

The complete per-file import audit accompanies the diagrams. This view selects the shared components used by the report screens.

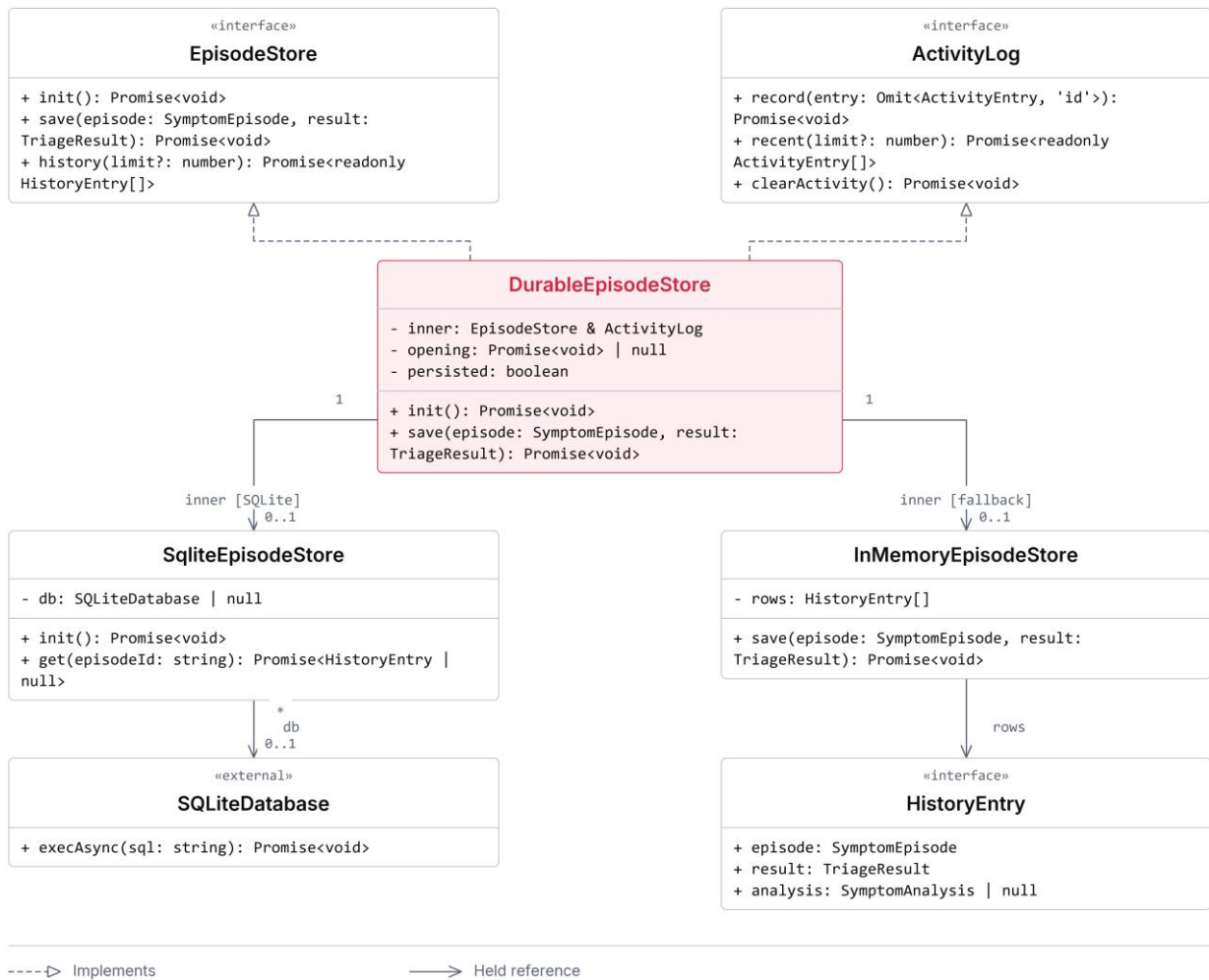
Figure 8b: View layer imports

The view imports show shared controls without turning screen functions into invented classes.

3.3 The class model

Figures 9–11 separate typed storage, triage and facility models. Associations show held references; dependencies show type use. Reverse arrows require reverse references in code.

Storage and persistence

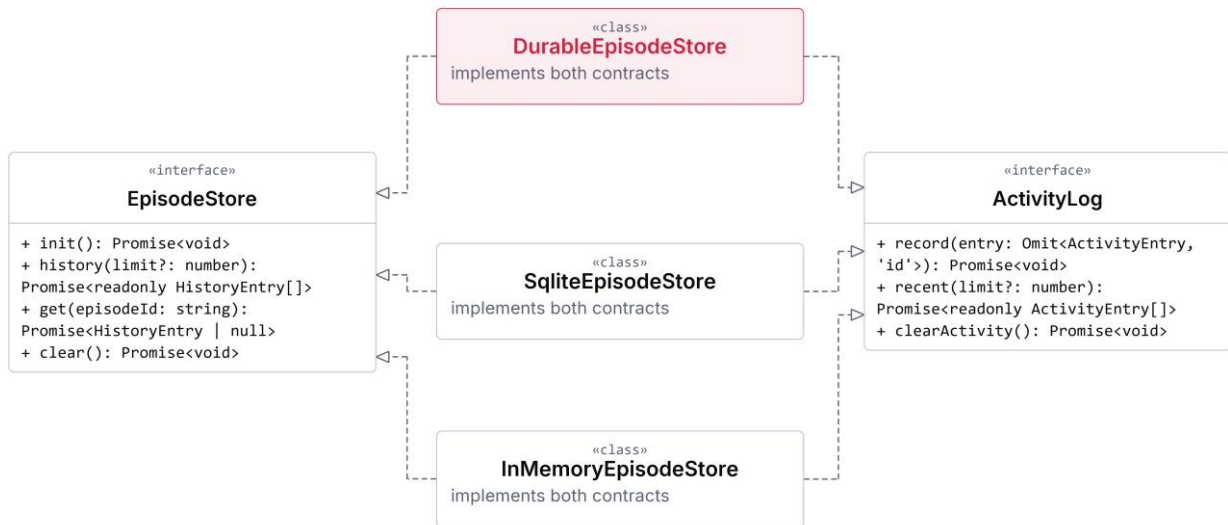


The two delegate associations are {xor}: exactly one active inner store. No lifecycle composition is inferred from a TypeScript property. All three stores implement both interfaces; Figure 9a shows all six realization links. HistoryEntry value types are expanded in the supplementary type catalogue.

Figure 9: Class diagram: storage package

DurableEpisodeStore has one active delegate, not simultaneous composition ownership of two stores. Figures 9a/9b expand contracts/history; 10a separates retained references from input/output types.

All storage interface realizations

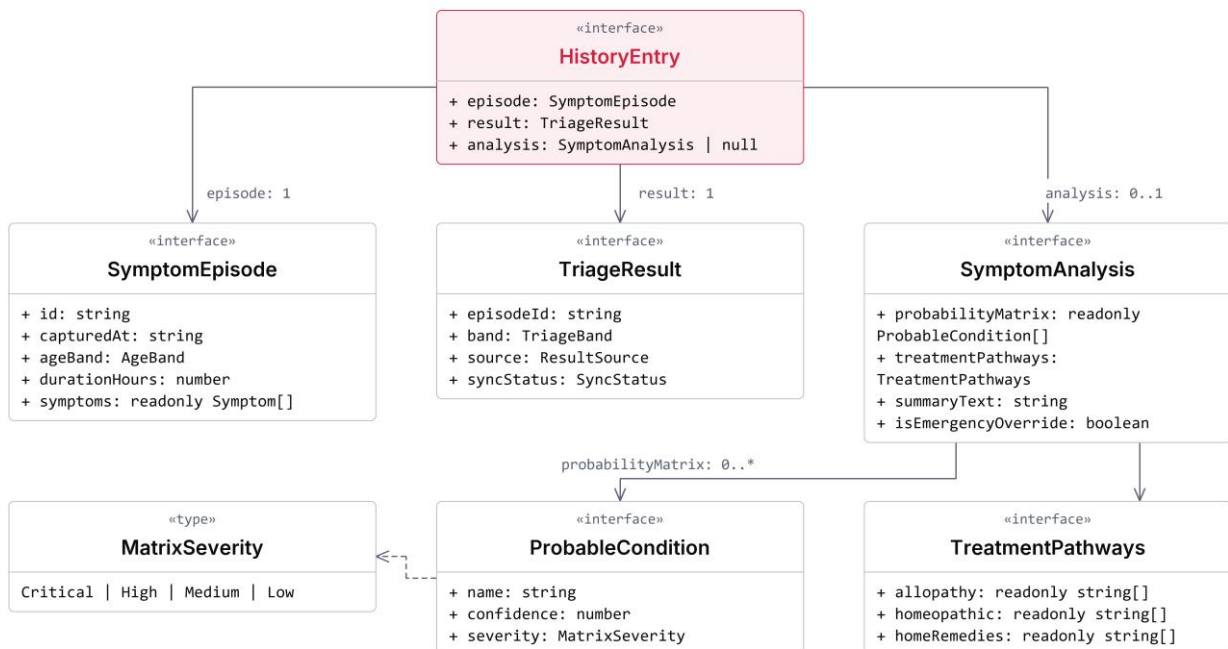


Every hollow triangle points to an implemented interface. These are six distinct realizations, not inheritance between storage implementations. The complete typed methods for both contracts and all classes are in the source catalogue; Figure 9 shows the active delegate references.

Figure 9a: All storage interface realizations

All three store classes implement both storage contracts; realization arrows are not inheritance between stores.

History entry value relationships

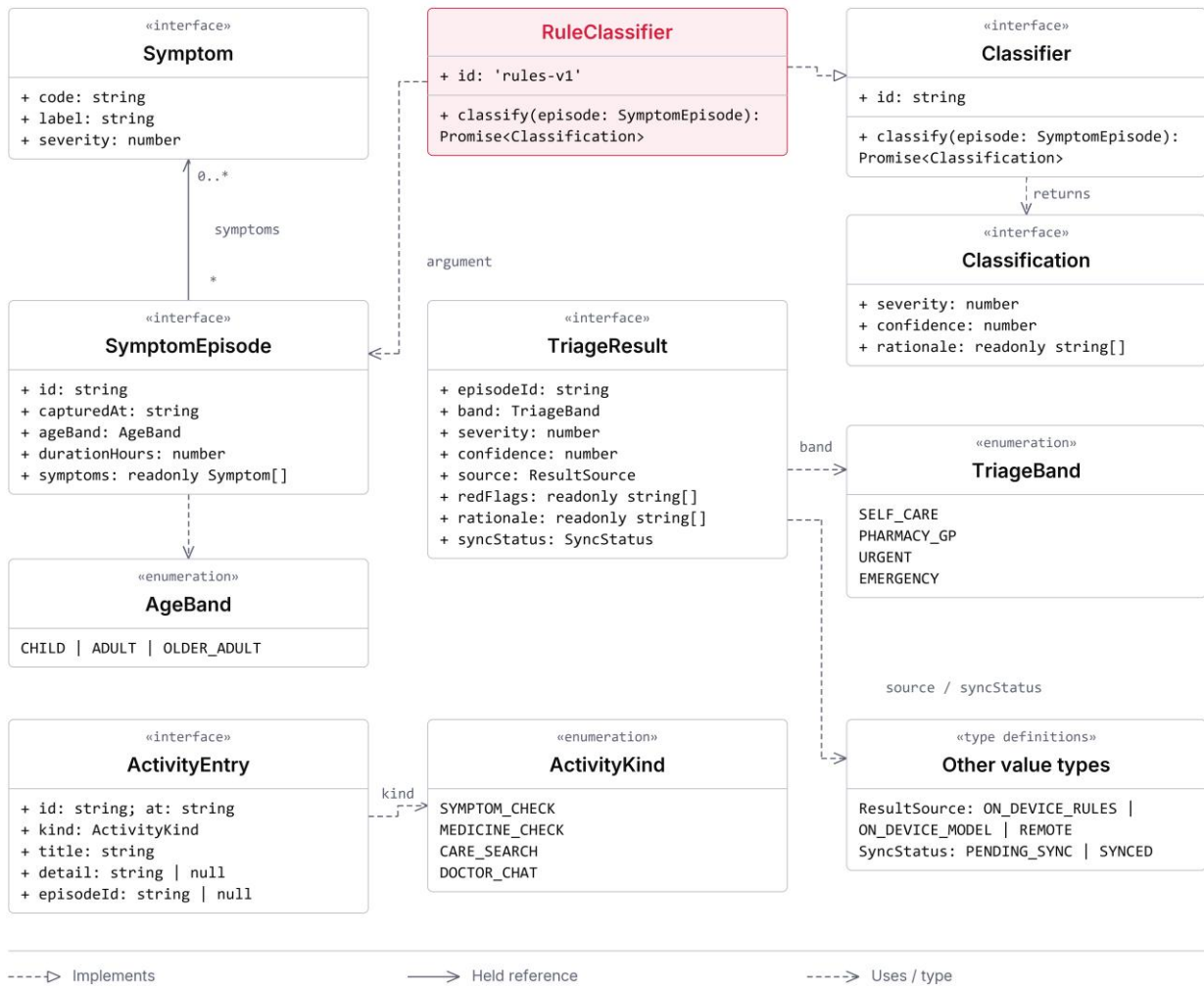


HistoryEntry retains episode, result and optional remote-analysis values. Remote analysis contains a condition list and treatment-pathway value. Figure 10 defines the remaining triage fields and their types. The source catalogue gives every locally declared class and complete interface signatures.

Figure 9b: History entry value relationships

HistoryEntry retains episode, result and optional analysis values, with their multiplicities shown.

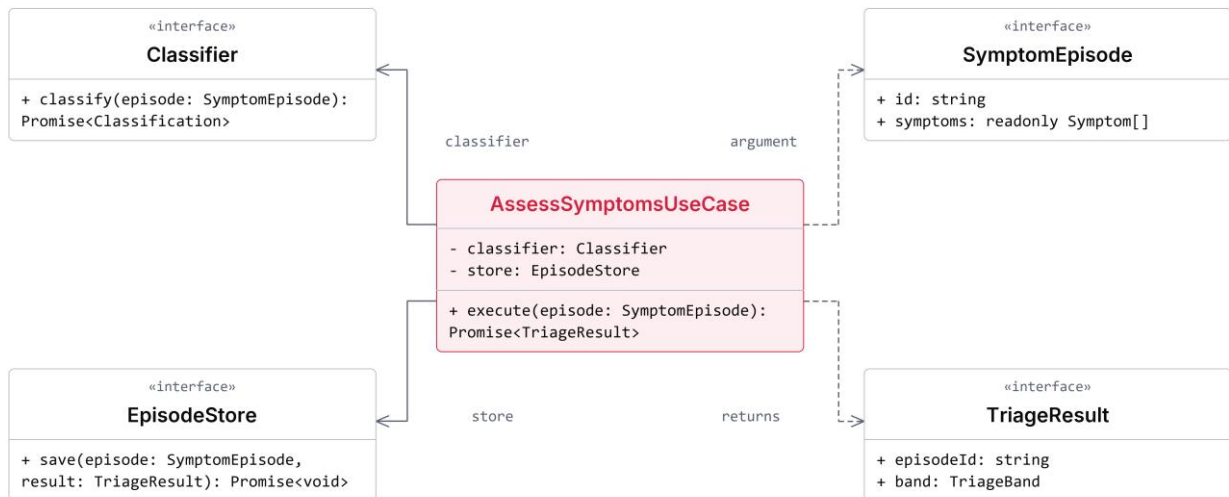
Triage and activity domain model



RuleClassifier is a data adapter shown here beside its domain contract. Classification = severity: number, confidence: number, rationale: readonly string[]. episodeId is an identifier, not a held object reference. No reverse association is invented. AssessSymptomsUseCase orchestrates the rule and red-flag logic (Figure 10a).

Figure 10: Class diagram: domain package

Assessment use case relationships

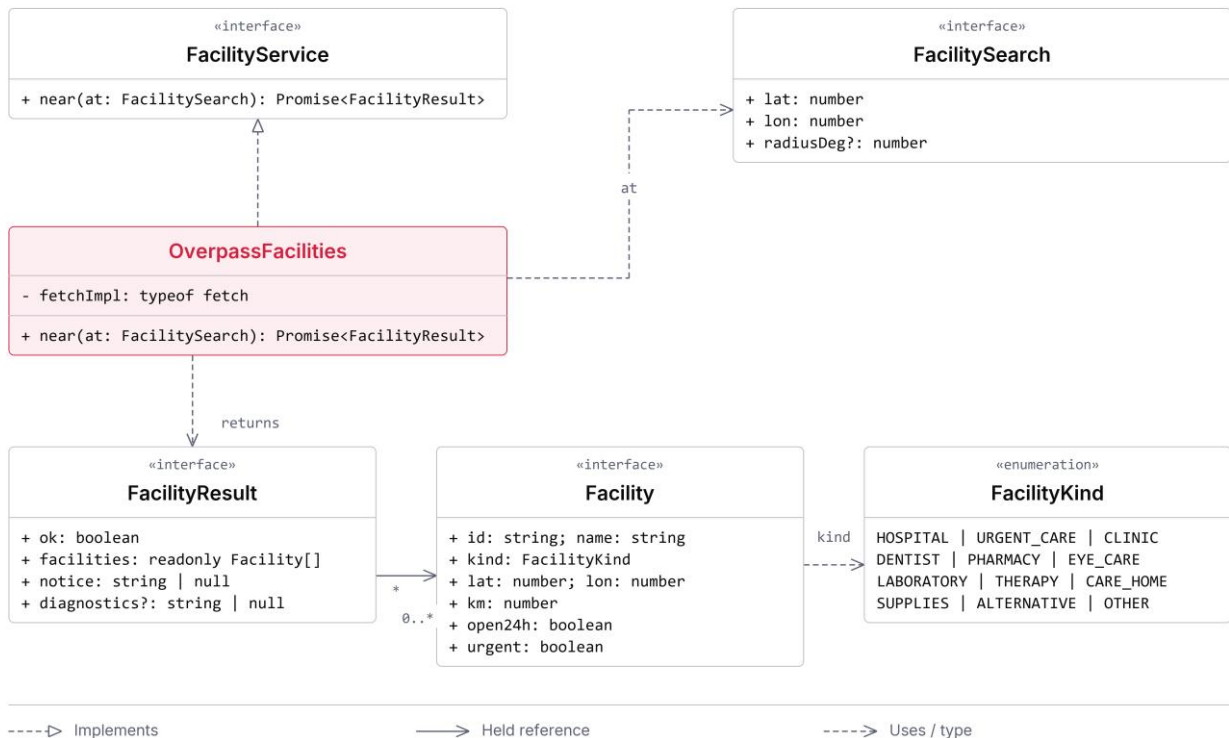


Solid open arrows show retained constructor references. Dashed open arrows show argument and return type dependencies. The use case calls `detectRedFlags`, `classifier.classify`, `bandForSeverity` and `store.save`. Neither interface holds a reverse reference to this use case.

Figure 10a: Assessment use case relationships

The use case holds classifier and store references; input/output types are separate dependencies.

Facility search model



----> Implements

—> Held reference

-----> Uses / type

`near` uses `FacilitySearch` as an argument and returns `FacilityResult`; neither is a retained provider property. `FacilityResult` holds `0..*` `Facility` values. `FacilityKind` is a value type. Additional facility fields are listed in the source type catalogue.

Figure 11: Class diagram: care package

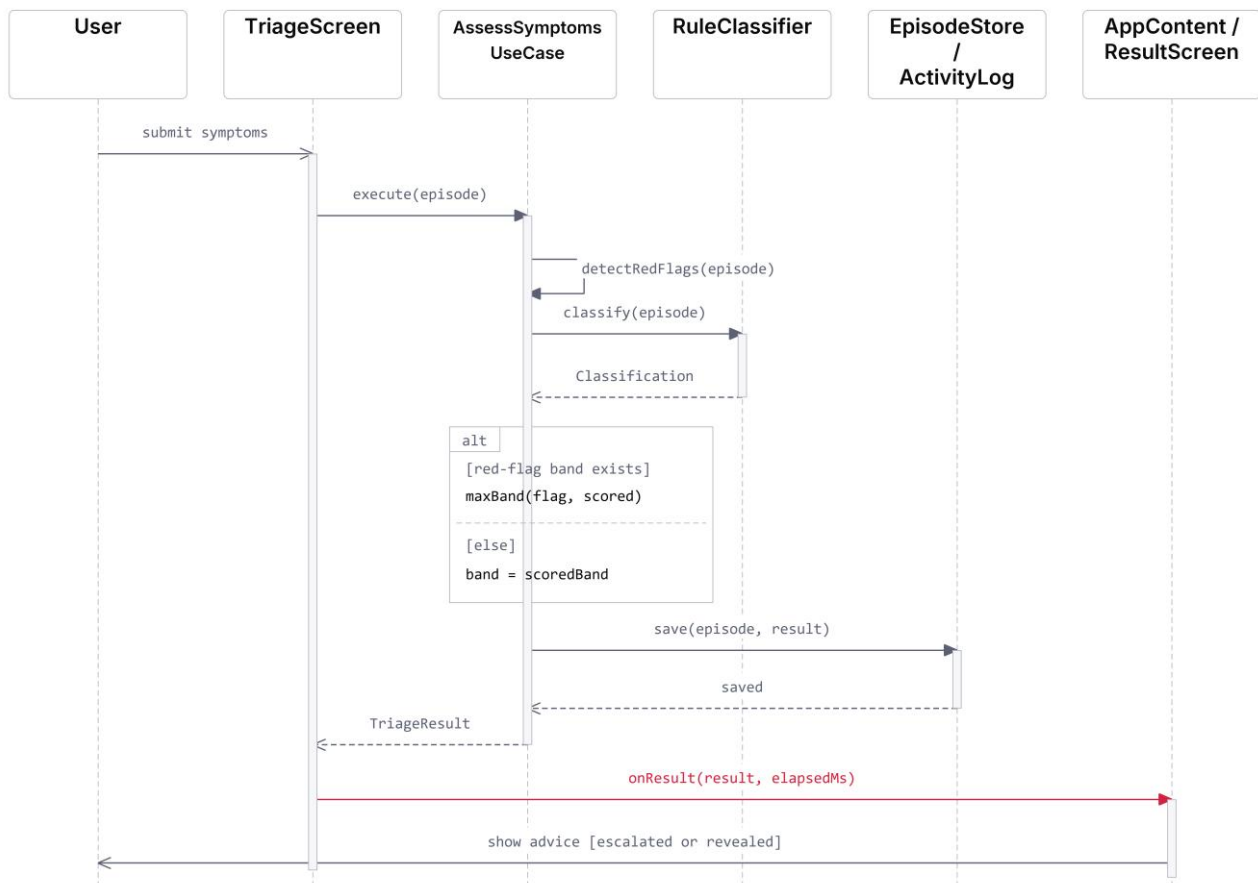
Figure 11 separates request, result envelope and facility values. A failed query differs from a successful empty response.

3.4 Runtime view

Figure 12 follows local assessment/save and user advice. Figures 12a/12b separate escalation from remote analysis. Stored classifier rationale is not rendered line by line; no rating is collected.

PULSEPOINT / ANDROID

Symptom check sequence

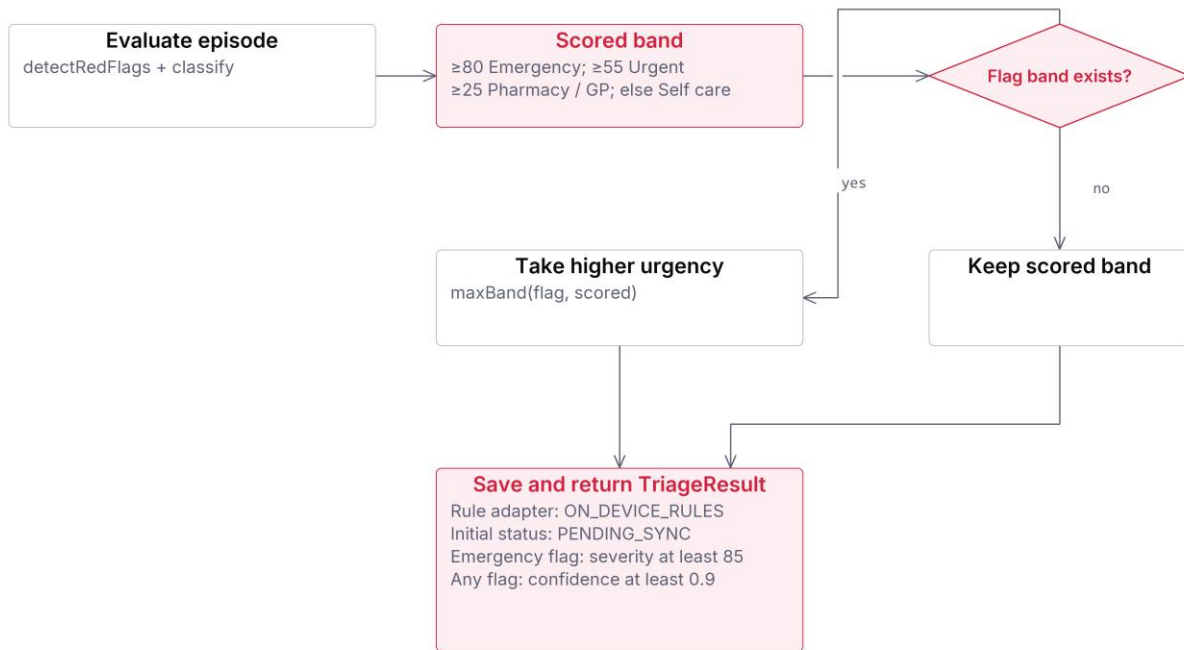


Filled solid = awaited call; dashed open = reply; solid open = asynchronous event. Activation bars show processing.

ResultScreen shows local band advice and red flags. Non-escalated results wait for remote completion or a 12-second reveal timer; see Figure 12b.

Figure 12: Symptom check sequence

Triage band and result status logic

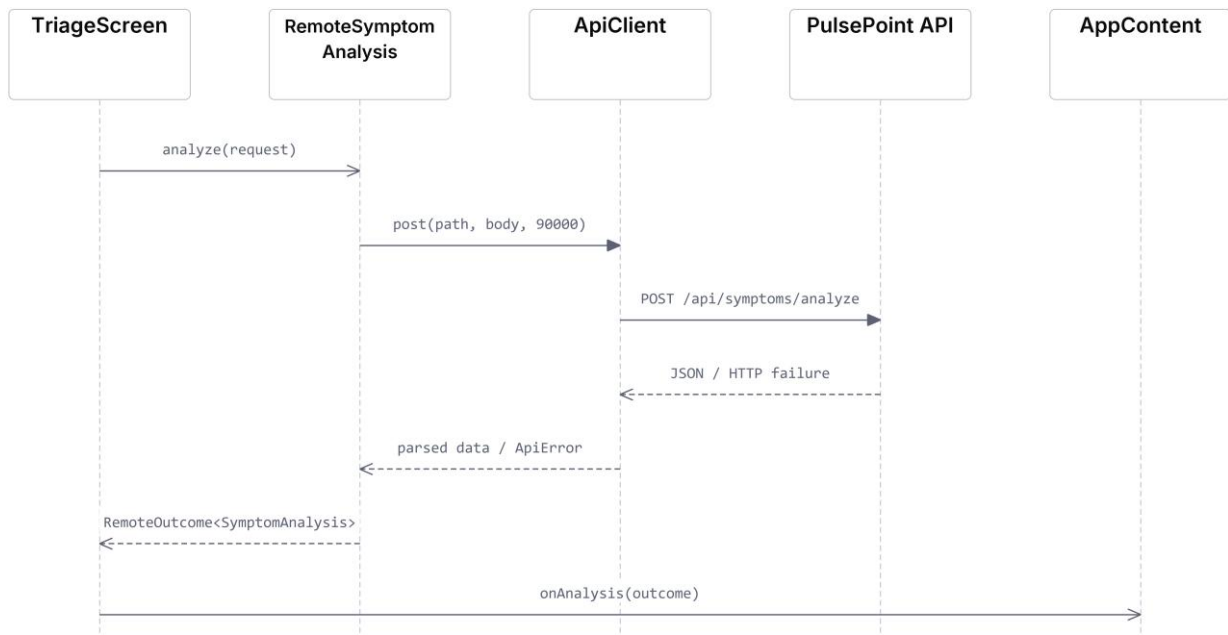


Severity thresholds are checked in descending order. A red flag cannot lower the scored urgency. PENDING_SYNC is a stored value. A background history-sync job is not implemented in this checkout.

Figure 12a: Triage band and result status logic

A red flag can raise urgency, while PENDING_SYNC is only a stored status, not proof of a sync worker.

Optional remote analysis sequence



This optional path starts after the local result callback. ApiClient obtains the Clerk session token through its injected provider and sends a bearer header. AppContent updates analysis state and may attach analysis to the current stored episode. Failure returns a typed notice. ResultScreen delays non-escalated display until analysis arrives or its 12-second timer expires.

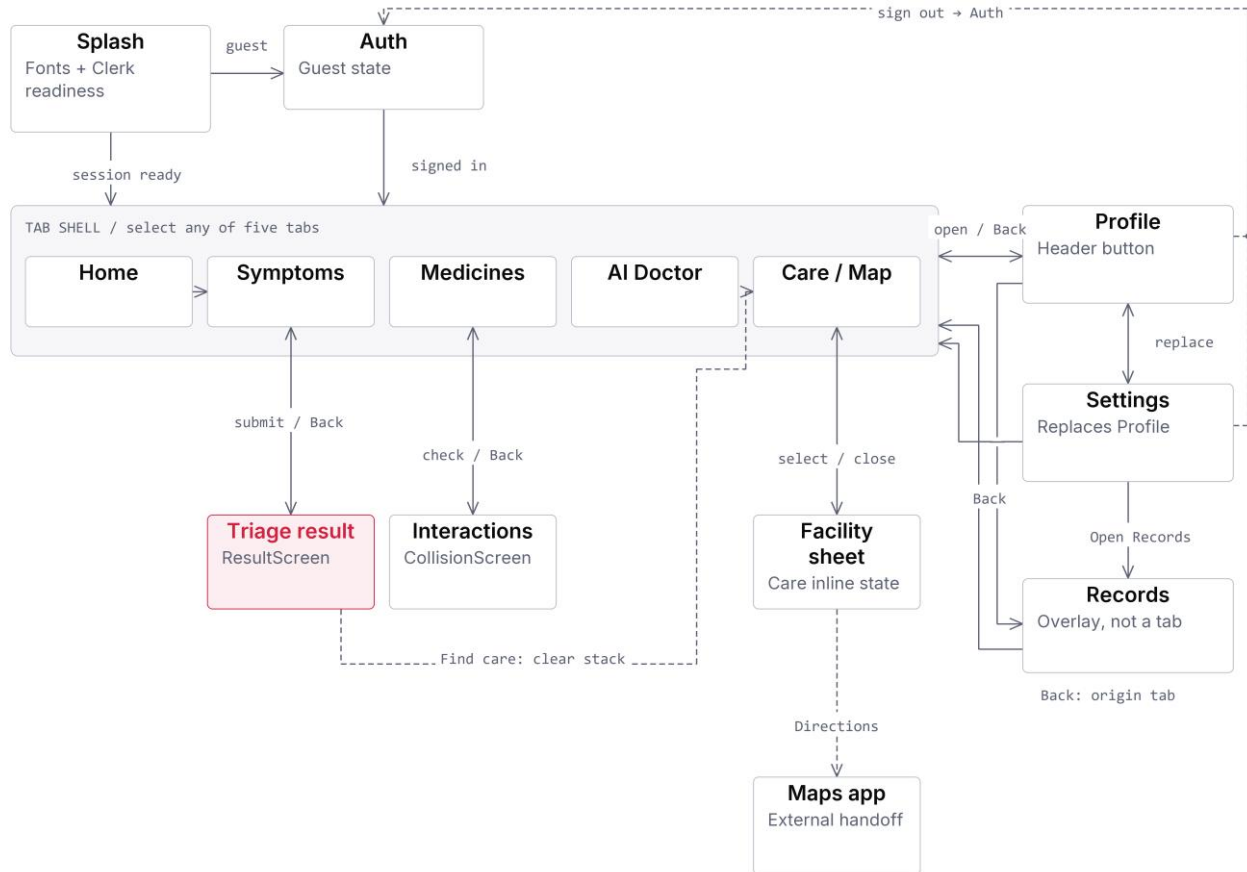
Figure 12b: Optional remote analysis sequence

The remote request sends symptom data separately from the local assessment and history store.

Figure 13 shows tabs and return routes: Find care clears the result stack; Records returns to its origin tab. Figures 13a/13b show query logic and coordinate-degree boxes.

PULSEPOINT / ANDROID

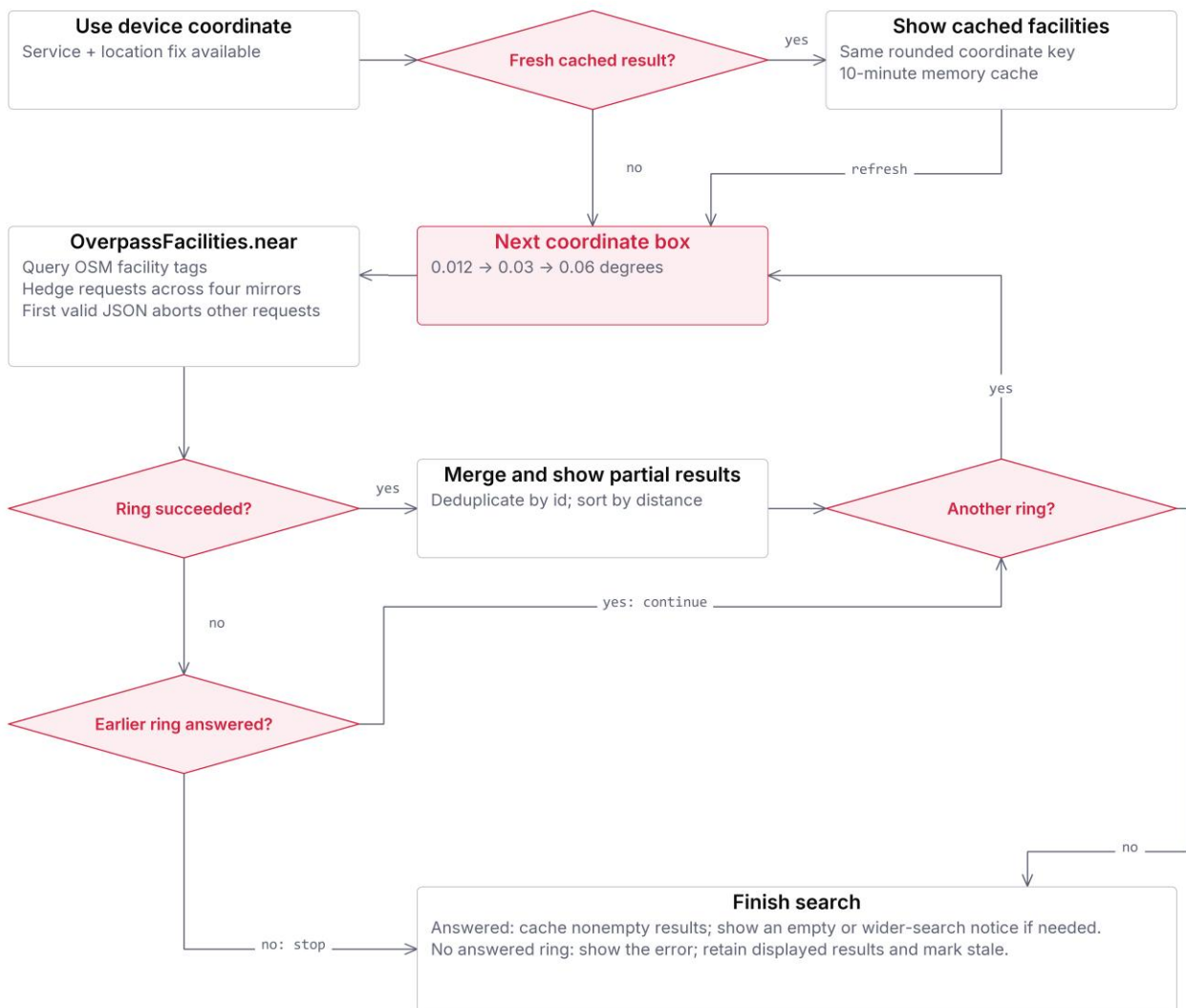
Screen navigation map



Double-headed = open / Back or reversible inline selection. Profile ↔ Settings replaces the overlay; Records Back returns to the origin tab. TabBar selects any tab and clears overlays; Home also has a Start check shortcut. Android Back pops an overlay, then returns a non-Home tab to Home. News and check-in overlay routes have no entry from the rendered tab controls. AI Doctor directly renders ChatScreen.

Figure 13: Screen navigation map

Facility API query flow

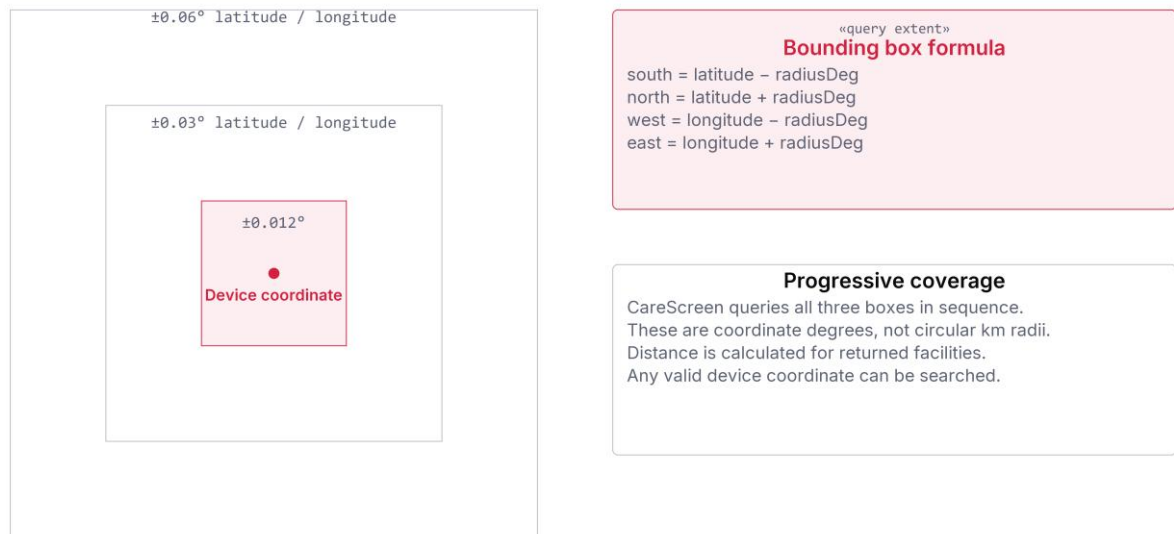


Warm results are shown immediately and refreshed. A first-ring failure stops the loop; after any successful ring, a failed wider ring allows the next attempt. PulsePoint queries an API, not HTML. GET network failure may trigger POST fallback; abort and an HTTP error alone do not. Outdated search runs are ignored.

Figure 13a: Facility API query flow

A first-ring failure stops the search; a later failure can still allow the next wider box.

Facility query geographic scope



The schematic is not a geographic map and is not drawn to physical distance scale. Longitude degree distance varies with latitude.
The repository has no hard-coded Auckland or New Zealand boundary. Results depend on OSM tags, data coverage and mirror availability.

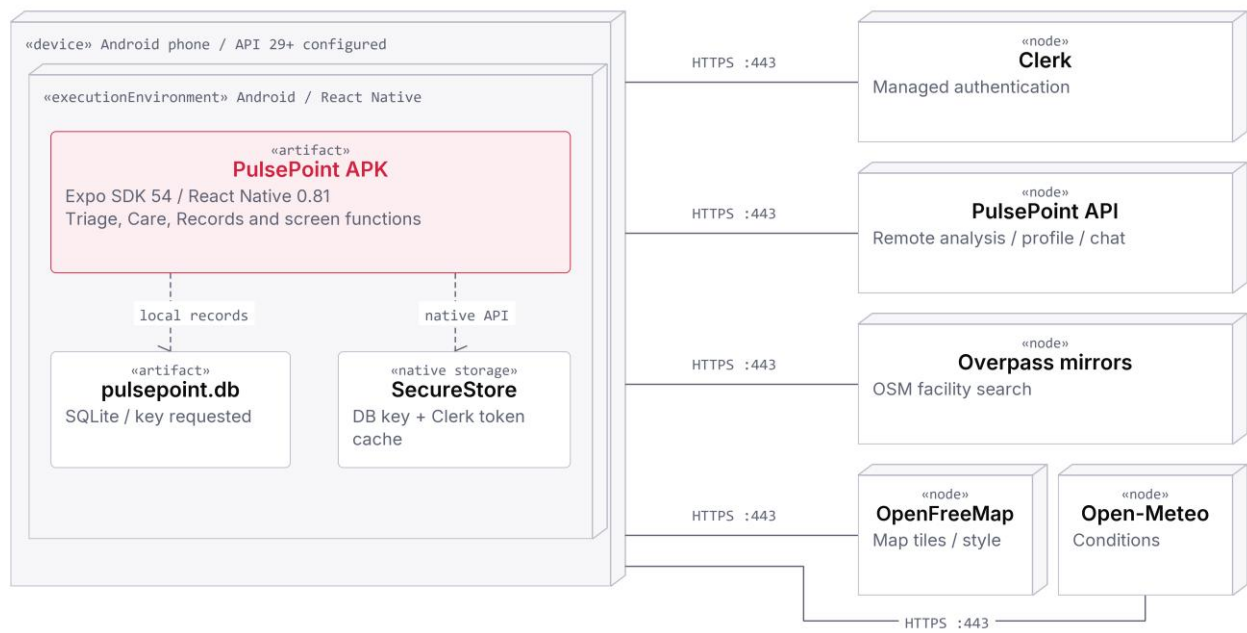
Figure 13b: Facility query geographic scope

The boxes show coordinate extent, not a fixed kilometre radius or a regional service boundary.

3.5 Physical view

Minimum API is 29. API 26 and a tested 2 GB RAM floor are not supported by the evidence. Memory, battery and touch reachability need hardware measurements.

Android deployment / current checkout

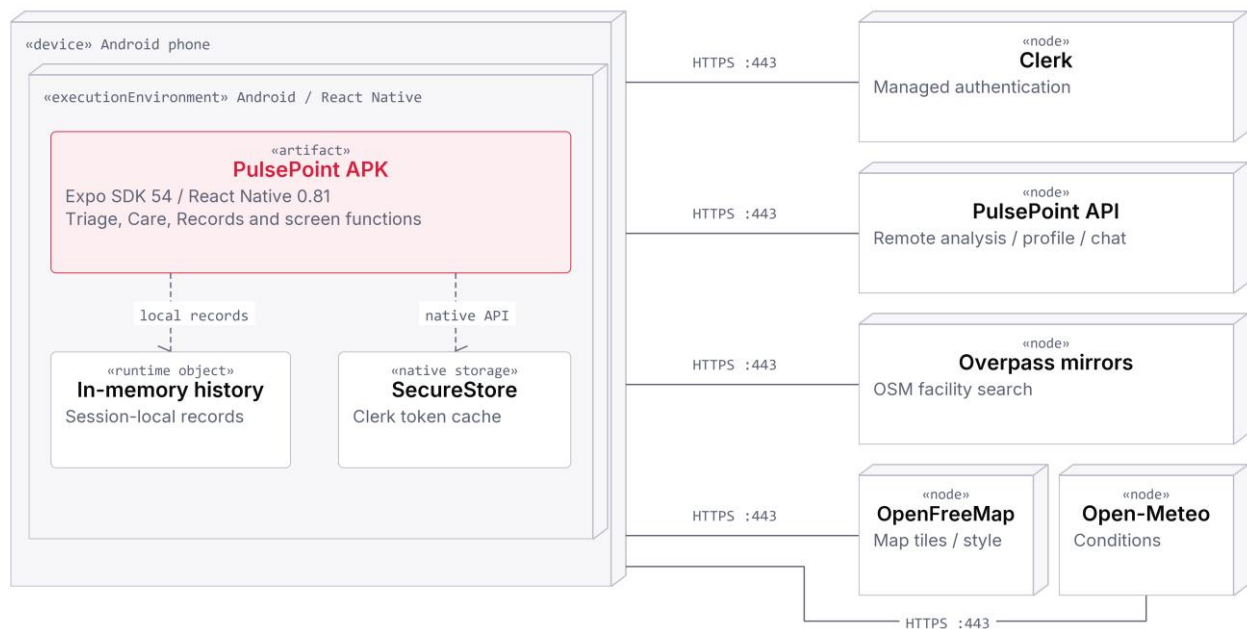


The local database and SecureStore are on the phone. PRAGMA key is requested, but SQLCipher is not enabled in the checked native configuration. Render / MongoDB details in the report are not verified by this Android repository. No direct phone-to-MongoDB connection is drawn.

Figure 14: Deployment view

Figure 14 separates local storage and external services. Figure 14a uses a real earlier snapshot; 14b shows shared API references; 14c shows schema. Local storage does not prevent remote symptom processing.

Android deployment / earlier Git snapshot



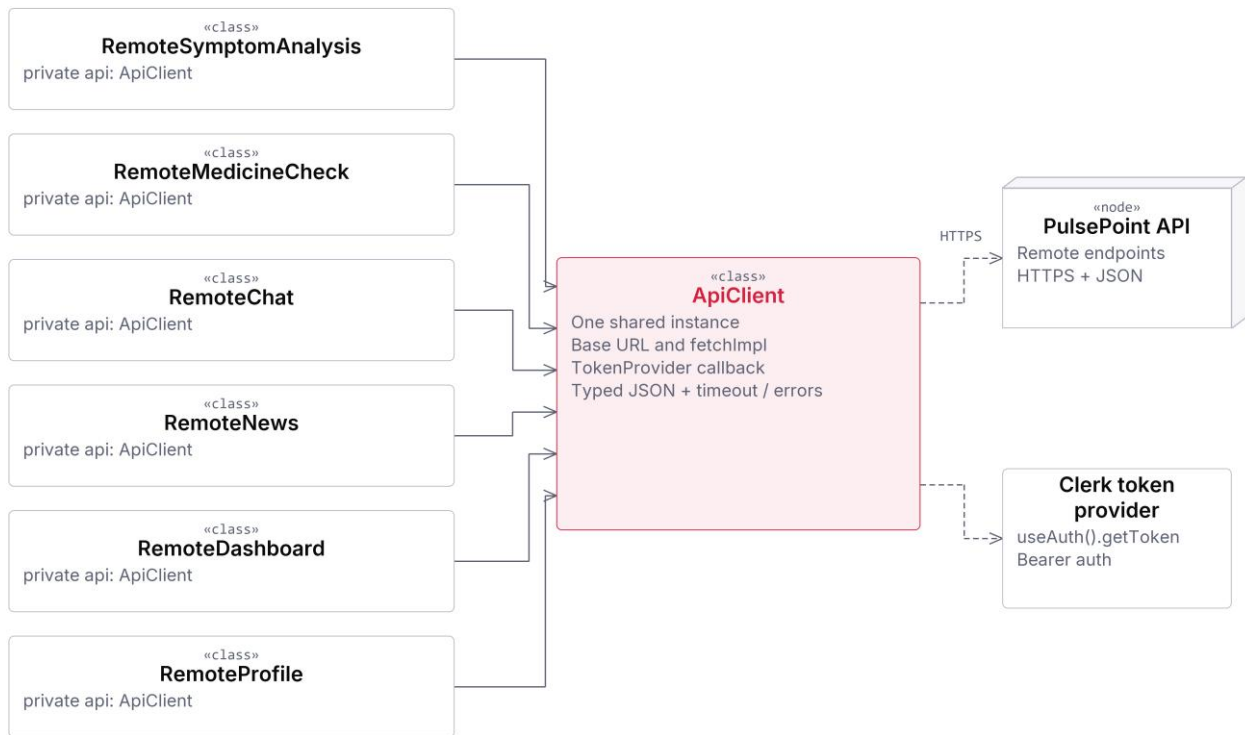
Old snapshot 961ebe3 (parent of persistence fix 931083a): AppContent constructs InMemoryEpisodeStore. Current checkout constructs DurableEpisodeStore.

Render / MongoDB details in the report are not verified by this Android repository. No direct phone-to-MongoDB connection is drawn.

Figure 14a: Earlier deployment from verified Git history

The old deployment records a specific Git snapshot, not an assumed Milestone 1 build.

Remote API adapter environment



Each adapter retains the same `ApiClient` instance supplied by `createServices`. No adapter-to-adapter references are implied. `OverpassFacilities` and `OpenMeteoConditions` use independent fetch integrations. Server-side database and hosting details cannot be established from this Android source.

Figure 14b: Remote API adapter environment

Six remote adapters share `ApiClient`; `Overpass` and `weather` use separate fetch integrations.

Local SQLite schema



Schema version 3 adds analysis_json. Episodes store JSON-valued symptom, red-flag and rationale fields; these are not separate SQL tables. The store issues PRAGMA key and saves its generated key in SecureStore. SQLCipher build support is not enabled in the checked configuration. Dashed id link is not a database-enforced foreign key. Runtime encryption of an installed binary has not been verified.

Figure 14c: Local SQLite schema

activity.episode_id is an application-managed link, with no declared SQL foreign-key constraint.

3.6 Variations from the Milestone 1 design

AppContent constructs RuleClassifier, replacing the proposed TensorFlow Lite approach. Its paths are inspectable and tested; this is not clinical validation or evidence of a deployed model.

Facilities changed from bundled data to Overpass. Cross-platform delivery was dropped: no iOS build or Apple-device test is included. This corrects the earlier stack-selection premise.

3.7 Implementation reflection

Development notes describe animation errors after Android view reattachment. JavaScript-driver conventions have source tests, but every transition still needs visual review.

The reported Overpass investigation compared hosts. GET is now first; POST fallback applies to eligible network failures. Per-host diagnostics distinguish transport failures from slow responses.

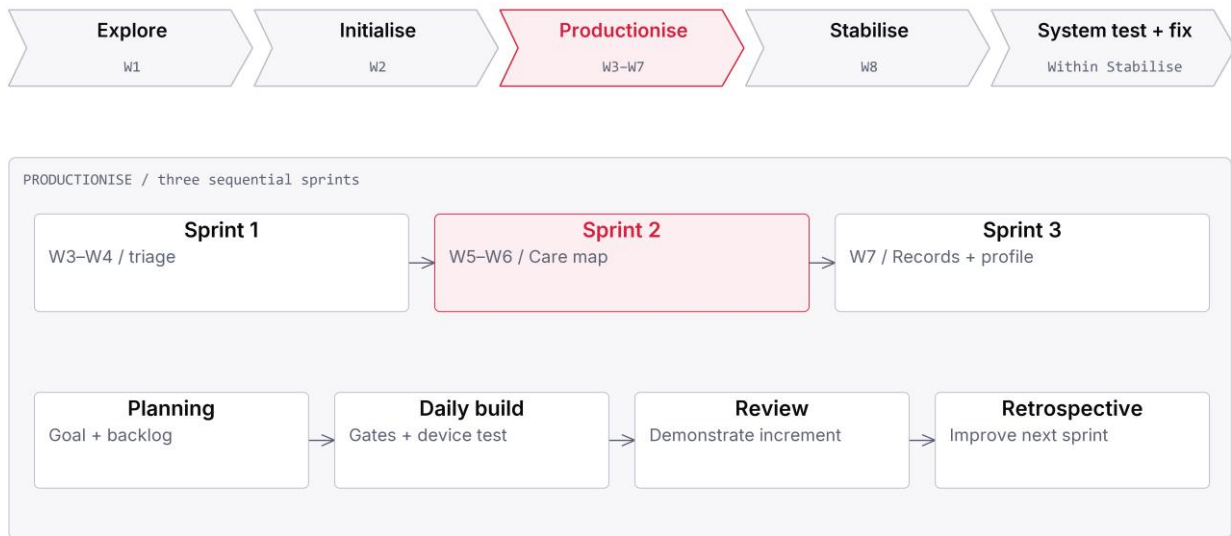
Location safeguards include accuracy fallbacks, recent cached fixes and mocked-reading rejection. Unit tests do not establish a measured hardware position-error bound.

The storage defect concerned assembly: memory was constructed despite an existing SQLite adapter. Git records the correction and tests check that choice. Native encryption verification remains missing.

3.8 Development process

Figure 15 puts Scrum-style iterations within Mobile-D phases (Abrahamsson et al., 2004; Schwaber & Sutherland, 2020). Figure 15a compares alternatives without claiming all were used.

Mobile-D phases with Scrum ceremonies



The phase strip gives project scope; the inset expands Productionise. System test and fix is folded into the Week 8 Stabilise phase.

A solo project adapts Scrum ceremonies; it does not claim a separate Scrum Team or Scrum Master. The schedule is a report plan, not a reconstruction of Git activity.

Figure 15: How Mobile-D and Scrum fit together

Figure 16 covers planning through retrospective; System test and fix joins Stabilise. No CI configuration was found, so checks on every commit are not claimed.

Development approach comparison

Dimension	Scrum	Rugby analogy	Kanban
Coordination	Sprint goal and product backlog	Whole-team collaboration across overlapping phases	Visualize work and control work in progress
Cadence	Fixed-length sprints, with review and retrospective	Historical holistic product-development approach	Continuous flow and explicit workflow policies
PulsePoint fit	Report adapts planning, review and retrospective to solo work	A comparison concept; not a named software framework in the app	A possible task-management alternative; not claimed as adopted
Report implementation	Mobile-D phases plus three sequential sprints	Phase overlap is not used to justify overlapping sprint dates	No measured flow metrics or WIP policy are supplied

Sources: Scrum Guide (2020), The New New Product Development Game (Scrum.org resource), The Kanban Guide (2025).

The comparison describes development methods, not Android runtime classes. It does not claim that all three methods were used.

Figure 15a: Development approach comparison

The comparison explains selection; it does not assert that every method was practised.

Figure 15b records recent commits, including mirror diagnostics and transport changes. It supports the account of iteration, without inventing sprint dates.

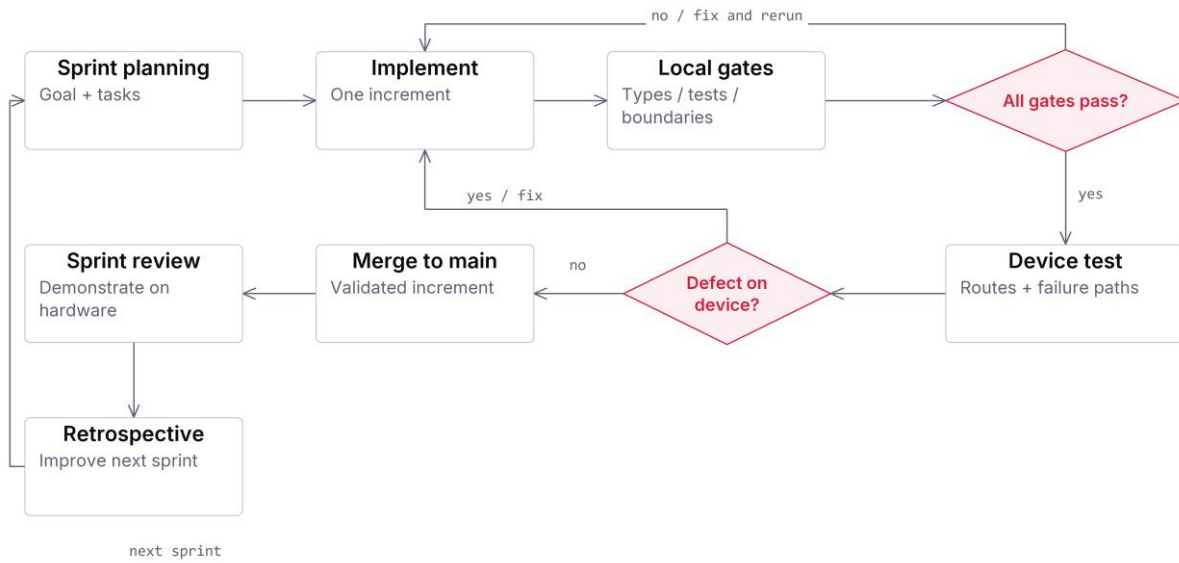
Verified Git change history

Label every place, and move labels rather than dropping them	5a8fa2b / 2026-10-01
Revert "Put the place names in pills"	4b0a802 / 2026-10-01
Put the place names in pills	f65565b / 2026-10-01
Draw pins from plain properties so the labels render	4e24c1b / 2026-10-01
Say when the wider search fails, and warm the first ring at launch	8fd0a42 / 2026-10-01
Keep widening after a ring fails, and label pins sooner	6e65225 / 2026-10-01
Ask Overpass over GET, and fall back to POST	5c8a1f1 / 2026-10-01
Report which mirror failed and how	14e3fb8 / 2026-10-01
Race the mirrors, keep the last good list, and fence the tests out of the bundle	2a6db08 / 2026-10-01
Make the map fail fast and answer sooner	1de930b / 2026-10-01
Fill the map in as results arrive	1250bda / 2026-09-30
Fix deleted records lingering, and show the analysis on a record	18caf0a / 2026-09-30

Extract: git log -12 --date=short --format="%h | %ad | %s"; checked on 1 October 2026.
Commits establish changes to the repository. They do not establish sprint boundaries, meetings, reviews or device-test outcomes.

Figure 15b: Verified Git change history

Build loop inside one sprint



Each decision has an explicit yes and no branch. Fixes return to implementation; only successful checks reach the review and retrospective. Local gates: TypeScript, test suite, domain import boundaries and contrast audit. These are the reported development steps, not an assertion that this editing task ran them.

Figure 16: The build loop inside one sprint

Figure 17 is an eight-week planning baseline, with sequential sprints, not reconstructed activity. The deadline is 23 October 2026. Figure 17a separates behaviour from outstanding checks.

Milestone 2 schedule



Sprint windows do not overlap: W3–W4, W5–W6 and W7. Parallel-looking task bars within one sprint indicate work allocation, not overlapping sprints. Planning baseline from the report dated 1 October 2026. This is not a measured commit timeline; the assignment deadline is 23 October 2026.

Figure 17: Milestone 2 schedule

Expected outcomes and evidence

Outcome	PulsePoint behavior	Evidence / remaining check
Explain urgency	Rule band, rationale and red flags are returned locally.	442 automated tests pass. These checks do not establish clinical accuracy.
Find nearby care	Progressive facility results, distance and maps handoff.	Live coverage depends on OSM and network. The report records a populated-area device check.
Retain records	Local episode and activity history; encryption is requested by the store.	Restart / delete checks reported. SQLCipher build support is unconfirmed; memory fallback is session-local.
Maintain the release	Typed contracts and automated development gates.	Type, boundary and 84 contrast checks pass. Device timing, cold start, memory and battery need measurements.

The local store requests an encryption key; native SQLCipher support has not been verified. Account history sync is not present. The result screen gives band advice and red-flag explanations. No user-rating feature is included.

Figure 17a: Expected outcomes and evidence

Expected outcomes are separated from the checks still needed to establish them.

3.9 Process reflection

I handled product scope, architecture, development, testing, UX and release. A team would distribute these roles; independent assembly review could have caught the storage defect.

Commits establish changes, not scheduled meetings. Add genuine board/review records where available. A process diagram cannot establish that its ceremonies occurred.

4 System Evaluation

4.1 Evaluation framework and benchmark devices

Milestone 1 criteria separate automated checks from reported device observations and missing measurements. E5 now tests a populated-area query; E6 is unverified.

Verification on 1 October 2026 returned 442 passing tests across 23 suites, plus type/boundary checks and 84 contrast pairs. Jest warned about an open handle. Earlier device notes lack reproducible benchmark logs.

4.2 Evaluation against functional criteria

ID	Criterion	Target	Evidence
E1/E2	Rule bands and escalation	All tested cases	442 tests pass; no clinical-accuracy claim.
E3	Direct forbidden domain imports	Zero matches	Boundary check passes.
E4	Typed data contracts	Type-safe interfaces	TypeScript passes; not a full architecture proof.
E5	Populated-area facility query	Non-empty result	Earlier report: 51 facilities; device log needed.
E8	Delete and reopen records	Deletion persists	Store tests pass; native restart check needed.
E10	Walkthrough stability	No unhandled crash	Device walkthrough evidence still required.

Table: Source-based heuristic review

Heuristic	Source finding / remedy	Priority
1 Status	Non-urgent Result can wait 12 s after local completion; show local advice first.	2
2 Familiar language	Records says encrypted despite unverified support; correct the assurance.	3

Heuristic	Source finding / remedy	Priority
3 Control	Back and deletion confirmation exist; no undo after deletion.	2
4 Consistency	Local-only and encrypted labels omit remote processing/fallback limits.	3
5 Prevention	Free-text-only input is accepted, but local rules receive no selected symptoms.	3
6 Recognition	Symptoms and band advice are visible; map legend/reachability needs device review.	Not rated
7 Efficiency	Find care shortcut exists; 12 s reveal wait slows the non-urgent path.	2
8 Minimal design	Result includes several analysis sections; clutter cannot be judged from source.	Not rated
9 Recovery	Care has retry; Records load errors lack a displayed catch/retry path.	2
10 Help	Confidence explanation exists; remote-processing disclosure needs review.	3

Priorities 0–4 are source-review judgements, not measured failure rates. The four-screen matrix records evidence and limits; no touch interaction or participant outcome is claimed.

4.3 Evaluation against non-functional criteria

Reported 77–91 ms and 133 MB results need logs and build identifiers. The app times assessment plus saving with Date.now(). Nielsen (1993) informs responsiveness; 500 ms/150 MB remain project targets requiring protocols.

84 colour pairs pass, not complete WCAG conformance. One-handed use, screen readers, font scaling, RAM, battery and cold start need device measurements.

The store generates a key and issues PRAGMA key, but SQLCipher is not configured in the checked build (Expo, n.d.). E6 is unverified. Credential tests are not a complete security audit.

Retain the APK and size command. Per-ABI builds/dependency removal may reduce size; measure the effect. Native release currently uses debug signing.

Table: Performance testing evidence

Metric	Protocol / target	Evidence
Local assessment	500 ms project target; separate rule and storage timing	Earlier report: 77–91 ms; reproducible log needed.
Release APK	150 MB project cap; record build and size command	Earlier report: 133 MB; retained APK needed.

Metric	Protocol / target	Evidence
Facility search	Time first and complete results; record mirrors/network	No reproducible timing result.
Cold start	Repeated release launches on named devices	Not measured.
Memory / battery	Profile triage and map use on hardware	Not measured.
Contrast	Specified pairs in both themes	84/84 pass; not full WCAG conformance.

Record build, device, repetitions and network conditions. Missing measurements remain missing.

4.4 Industry evaluation methods

ISO-based quality assessment and Nielsen source inspection were applied to available evidence. The latter is a limited code review, not an independent expert walkthrough or participant study.

ISO/IEC 25010:2011 organises quality categories; 25040 orders purpose, measures and decisions. Rule/type/boundary/contrast checks supply evidence. Encryption is unverified; native reliability and efficiency measurements remain incomplete.

Ten heuristics were checked against Triage, Result, Care and Records (Nielsen, 1994; Moran & Gordon, 2023). Findings include delayed advice, misleading encryption text, free-text/local-rule mismatch and no deletion undo. No app fix is claimed.

SUS remains planned (Brooke, 1996). Participants should check symptoms and find a pharmacy, then complete the standard questionnaire. No participant count, score or satisfaction result is supplied.

4.5 Reflection on the evaluation results

Automated checks cover rules and conventions better than native configuration or user behaviour. Source inspection also finds misleading labels and delays. Hardware evaluation should test those findings before their priorities are treated as settled.

5 Conclusion and Recommendations

5.1 Strengths

- Automated tests cover rule paths and escalation. Rationale is stored but not fully displayed.
- Typed interfaces separate assessment, storage and facilities; the boundary check covers direct forbidden imports.
- Records are local and colour pairs pass in both themes. These do not establish complete privacy or accessibility.
- Device-free tests support fast checks; the assembly defect shows why native verification remains necessary.

5.2 Limitations

- Android only; no iOS support version or evaluation is claimed.
- Clinical accuracy is unverified. Published triage cases and clinician review remain necessary.
- Remote services need connectivity; the memory cache is not a durable offline facility dataset.

- Source review cannot settle screen-reader behaviour, touch reachability or crowded map labels. No device usability/SUS study was completed.
- Encryption support is unverified. Memory fallback loses records on restart. Remote symptom processing limits the local-storage privacy claim.
- Benchmark records, production signing and low-end-device budgets are missing. Unreachable routes are not delivered features.

5.3 Future work

- Commercial agile teams should extend their definition of done to cover application assembly and native configuration. The memory-store defect passed unit tests because the real adapter choice was not checked. Each swappable adapter needs a build-level check, a device recovery test and a named reviewer. This applies to future apps that combine local models, storage and remote services.
- A security build check should report whether encryption support is compiled in, reject an unsupported configuration and test reopening data with the wrong key. This would turn the current PRAGMA-key assumption into observable evidence. A release gate should also check that privacy notices match the data sent by each service.
- The Overpass investigation supports a need for per-host transport and timeout diagnostics in mobile tooling. Future clients should record which attempt failed, with symptom text excluded from logs. Teams can then choose retries from observed failure patterns instead of treating every delay as a slow connection.
- Animation tooling needs tests across detach, reattach and background transitions, not just snapshots of a mounted view. A lifecycle test harness would address the reported native-driver defect before device review. It should accompany SDK upgrades and be assessed against the current workaround.
- Offline care and history recovery need explicit product decisions. A commercial team should test first-run offline behaviour, stale facility warnings and account recovery before adding sync. These changes require a merge strategy, data-retention rules and consent for remote processing; a new endpoint alone is insufficient.
- The next iteration should test the heuristic findings on hardware, conduct SUS, retain performance logs and obtain clinician review. Their results should set the next backlog. This connects development decisions to observed quality and user needs rather than adding features without checking their effect.

5.4 Conclusion

PulsePoint supplies an Android prototype and passing automated checks. Encryption, participant usability and device measurements remain unverified. Screenshots and demonstration evidence must be added before submission.

Future development should test what the shipped app assembles and tells users. This would have exposed memory-store selection and the unsupported encryption assurance earlier.

References

Abrahamsson, P., et al. (2004). Mobile-D: An agile approach for mobile application development. OOPSLA Companion, 174–175. ACM.

- Brooke, J. (1996). SUS: A quick and dirty usability scale. In P. W. Jordan, B. Thomas, B. A. Weerdmeester, & I. L. McClelland (Eds.), *Usability evaluation in industry* (pp. 189-194). Taylor & Francis.
- ISO. (2011). ISO/IEC 25010:2011: System and software quality models.
- ISO. (2011). ISO/IEC 25040:2011: Evaluation process.
- Martin, R. C. (2017). *Clean architecture: A craftsman's guide to software structure and design*. Prentice Hall.
- Nielsen, J. (1993). *Usability engineering*. Academic Press.
- Nielsen, J. (1994, reviewed 2024). 10 usability heuristics for user interface design. Nielsen Norman Group.
<https://www.nngroup.com/articles/ten-usability-heuristics/>
- OpenStreetMap Foundation. (2026). Overpass API documentation. https://wiki.openstreetmap.org/wiki/Overpass_API
- Moran, K., & Gordon, K. (2023). How to conduct a heuristic evaluation. Nielsen Norman Group.
<https://www.nngroup.com/articles/how-to-conduct-a-heuristic-evaluation/>
- Schwaber, K., & Sutherland, J. (2020). *The Scrum Guide*. <https://scrumguides.org>
- World Wide Web Consortium. (2023). Web Content Accessibility Guidelines (WCAG) 2.2.
<https://www.w3.org/TR/WCAG22/>
- Expo. (n.d.). SQLite, SDK 54: SQLCipher configuration. <https://docs.expo.dev/versions/v54.0.0/sdk/sqlite/>
- Takeuchi, H., & Nonaka, I. (1986). The new new product development game. Scrum.org resource.
<https://www.scrum.org/resources/new-new-product-development-game>
- Kanban Guides. (2025). The Kanban guide. <https://kanbanguides.org/the-kanban-guide/>
- Word count: 3,552